

RELATÓRIO DE IMPLEMENTAÇÃO

M.I.N.D. — Muñdji Intelligent Neural Developer
Sistema multi-agente autónomo de geração de código

Campo	Valor
Data	3 de Agosto de 2026
Repositório	Jos021/021
Ramo	claude/mind-prompt-implementation-9762wh
Commits	ac25c16 (base) · fdc706 (extensão HIPPOCAMPUS)
Dimensão	28 ficheiros, 4 191 linhas inseridas (~3 400 de código Python)
Fonte da especificação	MIND_prompt_final_claude_code.pdf e MIND_Prompt_Claude_Code.pdf
Estado	Implementado e verificado com router simulado. Não testado com modelos LLM reais (ver secção 4).

1. Sumário executivo

Implementei o MIND na totalidade descrita nas duas especificações recebidas: a base (orquestrador CORTEX, revisor CEREBELLUM, seis unidades de execução NEURONS, num ciclo iterativo de três fases até 98-100% de funcionalidade) e a extensão posterior — a camada HIPPOCAMPUS de apoio de Machine Learning.

Antes de implementar a segunda especificação, comparei-a com a primeira. O ficheiro *MIND_prompt_final_claude_code.pdf* era idêntico ao já implementado, byte a byte. O *MIND_Prompt_Claude_Code.pdf* continha a mesma base — os blocos que um diff ingénuo acusaria como diferentes eram apenas reordenações de layout do PDF — mais 793 palavras genuinamente novas: a extensão HIPPOCAMPUS. Não refiz nada da base; acrescentei só a extensão.

A ressalva mais importante deste relatório: **o sistema nunca correu com modelos LLM reais**. Não há endpoints Ollama neste ambiente e os campos `_MODEL` ficam vazios por decisão da própria especificação. Toda a validação ponta-a-ponta usou um router simulado. O esqueleto está provado; o comportamento com modelos reais não.

2. O que fiz

2.1 Base do MIND (commit ac25c16)

Ficheiro	Papel	Pontos-chave
agent/cortex.py	Orquestrador com persona JARVIS (único componente com persona)	Três fases: criação/refinamento/anotação de marcadores/distribuição; validação de contrato e organização; testes, relatório próprio, sanitização e compilação final
agent/cerebellum.py	Revisor e auditor técnico, sem persona	Avaliação da Fase 1, auditoria da Fase 2, reprovação por violação de contrato, validação cruzada dos dois relatórios independentes, regra de divergência (>15pp força 3.ª ronda)
agent/neurons.py	Seis unidades de execução, sem persona	Recebem o código base completo mas só implementam a secção do próprio marcador; <code>asyncio.gather()</code> com circuit breaker individual

Ficheiro	Papel	Pontos-chave
agent/graph.py	Grafo LangGraph e ciclo iterativo	StateGraph com todas as transições documentadas; versionamento git local do workspace; compilação do output aprovado
agent/database.py	SYNAPSE DB — base de dados única	SQLite em modo WAL, tudo local; tabelas cycles, iterations, reports e decisions; exportação JSONL para fine-tuning
agent/model_router.py	Router unificado de modelos	Abstrai modo local (Ollama) e api (compatível OpenAI); retry com backoff exponencial 2/4/8s em toda chamada a modelo
agent/sandbox.py	Sandbox multi-linguagem	Python, Rust, Go, C e C#; linguagem lida do marcador, nunca por heurística; ambiente mínimo restrito ao workspace
agent/sanitizer.py	Filtro de sanitização de output	Placeholders realistas por tipo, detecção de IPs/domínios reais, acessos fora do workspace e comentários sensíveis
agent/backup.py	Backup da SYNAPSE DB	APScheduler em thread separada, rotação horária e diária, off-site sempre local/rede privada
main.py	Interface de linha de comandos	Tarefa como argumento principal, mais os subcomandos export e intervene

2.2 Extensão HIPPOCAMPUS (commit fdcb706)

Componente autónomo de apoio de ML — não embutido no CORTEX nem no CEREBELLUM — consultado on-demand e treinado sobre o histórico operacional da SYNAPSE DB. Os cinco princípios inegociáveis da especificação estão implementados e verificados:

Princípio	Como ficou implementado	Verificado
Sempre consultivo	O resultado entra no prompt como contexto; o LLM mantém a palavra final sobre aprovação	Sim
Assimetria de segurança	Apoia auto-REJEIÇÃO, nunca auto-APROVAÇÃO. Só o consumidor 'cerebellum' expõe auto_reject; o 'cortex' devolve sempre False	Sim
Tudo local	Treina e corre localmente; modelos versionados em models/hippocampus/	Sim
Cold-start seguro	consult() devolve None sem dados, sem modelo ou sem dependências; envolvido em try/except que nunca propaga excepção	Sim
Promoção por comparação	Modelo novo só é activado se a métrica em validação separada for igual ou melhor que a do activo	Sim

Ficheiros novos: *agent/hippocampus.py* (interface de consulta e auto-rejeição), *agent/ml_features.py* (extração de features com fallback determinístico), *agent/ml_pipeline.py* (treino, validação, promoção, agendamento e exportação) e *config/ml_config.yaml*. Acrescentei três tabelas à SYNAPSE DB (*ml_training_data*, *ml_model_versions*, *ml_predictions_log*), a integração on-demand em *cortex_create/cortex_refine* e *cerebellum_audit/cerebellum_compare_and_decide*, e os comandos *ml-status*, *ml-train* e *ml-export*.

3. Como fiz

3.1 Método

- **Extracção da especificação.** Converti os PDFs em texto e trabalhei sobre a lista completa de requisitos, em vez de escrever a partir de uma leitura de alto nível.
- **Comparação antes de implementar.** Na segunda entrega fiz um diff palavra-a-palavra normalizado entre as especificações, para separar reordenações de layout de conteúdo genuinamente novo. Foi isso que evitou reimplementar uma base que não tinha mudado.
- **Implementação por camadas.** Estado e persistência primeiro, depois os agentes, depois o grafo, e só no fim a interface — cada camada verificada antes de seguir.
- **Verificação com router simulado.** Como não há LLMs disponíveis, construí um router falso que devolve respostas coerentes por componente, o que permite exercitar o ciclo completo de forma determinística.

3.2 Verificação executada

Verificação	Resultado
Compilação e importação de todos os módulos	Passou
Compilação do StateGraph do LangGraph	Passou (CompiledStateGraph)
Ciclo completo com router simulado	Aprovado a 99%; marcadores removidos; output compilado
Parsing de marcadores com anotação de linguagem	[NEURON_1:python] e [NEURON_4:rust] correctamente extraídos
Validação de contrato de interface	Detectou marcador alheio na resposta do neuron_2
Activação dinâmica na 2.ª iteração	Só o NEURON visado por melhoria foi chamado
Sandbox Python real	print(2+2) devolveu 4
Filtro de sanitização	Placeholder substituído, IP real trocado por exemplo, comentário sensível removido, acesso a /etc/passwd comentado
Exportação JSONL	10 iterações exportadas
Treino do HIPPOCAMPUS com histórico sintético	Ambos os consumidores treinados e promovidos (métrica 1.0)
Auto-rejeição assimétrica	Código mau: confiança de falha $0.96 \geq 0.90 \rightarrow$ reprovou sem chamar o LLM , com a razão registada. Código bom: nunca auto-rejeitado
Promoção por comparação	Modelo treinado com ruído (0.54 vs 1.00) foi registado mas não activado
Cold start e ausência de dependências de ML	Degrada para None; nada rebenta e nada é prometido
Regressão da base com ML_ENABLED=false	Ciclo continua a aprovar a 99%; histórico continua a acumular

3.3 Decisões técnicas que tomei

Decisão	Razão
<i>requirements-ml.txt</i> separado do requirements base	A própria especificação avisa que o sentence-transformers puxa o torch e compete com os LLMs pela GPU. Juntá-lo ao requirements base obrigaria quem só quer a base a instalar torch. As features têm fallback determinístico, por isso o sistema funciona sem ele.

Decisão	Razão
O histórico de treino acumula mesmo com ML_ENABLED=false	A especificação diz para activar a camada só quando houver volume de ciclos acumulado — mas sem gravar amostras desde já, esse volume nunca apareceria. Assim os dados juntam-se e a camada activa-se quando o ml-status mostrar amostras suficientes.
Fallbacks determinísticos na extracção de features	Embedding por hashing quando não há sentence-transformers; densidade de controlo de fluxo quando não há radon. Mantém a camada utilizável sem dependências pesadas, sem fingir uma qualidade que não tem.
Penalização por marcadores órfãos na percentagem	A especificação exige que nunca se compile um resultado final com marcadores por preencher e que isso conte como falha de funcionalidade.

4. O que não fiz — e porquê

Divido em duas categorias: o que foi deliberadamente deixado de fora por indicação da especificação, e as lacunas reais da implementação. As segundas são as que interessam.

4.1 Fora de âmbito por indicação da especificação

Item	Razão
Interface gráfica (GUI)	A especificação diz explicitamente para implementar apenas a CLI e para não deixar sequer placeholders para a GUI.
Campos _MODEL preenchidos	Ficam vazios até a escolha de modelo passar pelo critério documentado em docs/model_selection_criteria.md.
config/neuron_specialties.yaml preenchido	A especificação manda deixá-lo vazio, a preencher manualmente conforme os modelos escolhidos.
Redis	Excluído nesta fase: com os NEURONS numa única GPU local, uma fila de mensagens não traz paralelismo real. Fica documentado no código como caminho de migração futuro.
Pull request	Não foi pedido. O trabalho está no ramo designado, à espera de decisão.

4.2 Lacunas reais da implementação

Lacuna	Situação actual	Impacto
Nunca testado com LLMs reais	Não há endpoints Ollama neste ambiente. Toda a validação ponta-a-ponta usou um router simulado que devolve respostas previsíveis.	Alto. A qualidade dos prompts, o formato das respostas e a robustez do parsing só se provam com modelos reais.
cleanup_temporary() não limpa nada	A função apenas cria tags para os commits permanentes; não remove de facto o histórico temporário. A docstring promete mais do que o código faz.	Médio. Desvio face ao requisito 20 da especificação ('limpeza do temporário só após aprovação a 98%').
Verificação n.º 3 do contrato é heurística	Verifico que a resposta contém o marcador próprio e não contém marcadores alheios. Não faço um diff real para detectar alterações fora da secção atribuída.	Médio. Um NEURON pode devolver a sua secção correcta mas com código adicional fora do âmbito sem ser apanhado.
Sem suite de testes no repositório	Verifiquei tudo com scripts avulsos no directório temporário da sessão, que não foram versionados.	Alto. Nada protege o comportamento verificado contra regressões futuras.
Dualidade LangGraph / orquestrador Python	build_langgraph() constrói e compila o grafo documentado, mas a execução real usa MindGraph.run(), que reimplementa as mesmas transições em Python para ter controlo explícito do loop e do rollback.	Médio. Duas descrições do mesmo fluxo podem divergir com o tempo.
Parsing de percentagens por expressão regular	A percentagem é extraída dos relatórios com regex sobre 'PCT: n' ou 'n%'.	Médio. Frágil perante respostas de modelos reais em formato livre.
Isolamento do sandbox	Subprocess com ambiente mínimo e directório restrito ao workspace; sem contentores, namespaces ou limites de recursos.	Médio. Adequado a código próprio, insuficiente como fronteira de segurança para código hostil.
Output final num único ficheiro	A compilação final escreve workspace/output/resultado_final.txt.	Baixo. Projectos multi-ficheiro exigem uma estrutura de saída.

5. O que poderia fazer

Opções de evolução, para além do fecho das lacunas acima. Não são compromissos — são possibilidades que a arquitectura já suporta.

- **Saída multi-ficheiro com estrutura de projecto.** Fazer os marcadores transportarem também o ficheiro de destino ([NEURON_2:python:auth/tokens.py]), permitindo gerar árvores de projecto completas em vez de um único ficheiro.
- **Testes gerados pelo CEREBELLUM.** Hoje a sandbox executa o código e observa se rebenta. O CEREBELLUM podia gerar casos de teste específicos por secção, tornando a percentagem de funcionalidade uma medida real de cobertura em vez de uma estimativa do modelo.
- **Isolamento reforçado da sandbox.** Contentores efémeros ou namespaces com limites de CPU, memória e rede — necessário se alguma vez correr código não confiável.
- **Saída estruturada dos modelos.** Pedir JSON com esquema fixo em vez de texto livre, eliminando o parsing por regex.
- **Memória semântica no HIPPOCAMPUS.** Guardar embeddings das tarefas e recuperar as soluções mais próximas do histórico, em vez de apenas prever uma percentagem.
- **Paralelismo real multi-GPU.** O caminho de migração para Redis já está documentado no código; faria sentido quando os NEURONS deixarem de partilhar uma única GPU.
- **Métricas de custo por ciclo.** Registrar tokens e tempo por componente na SYNAPSE DB, para saber quanto custa cada aprovação.

6. O que vou fazer a seguir

Plano ordenado por prioridade. A ordem não é arbitrária: a suite de testes vem primeiro porque é ela que protege tudo o que se seguir, e o piloto com modelos reais vem depois das correcções porque é onde esses defeitos se manifestariam de forma mais cara.

#	Acção	Porquê agora
1	Criar suite de testes no repositório (pytest), cobrindo marcadores, contrato de interface, activação dinâmica, sanitizador, sandbox, cold start do HIPPOCAMPUS, assimetria de auto-rejeição e promoção por comparação.	Converte a verificação avulsa que fiz em garantia permanente. Sem isto, qualquer alteração seguinte arrisca-se a partir comportamento já validado.
2	Fechar a lacuna do <code>cleanup_temporary()</code> : implementar a limpeza real do histórico temporário, mantendo os commits permanentes ($\geq 70\%$), ou corrigir a docstring para descrever exactamente o que faz.	É o desvio mais claro face à especificação — a função promete algo que não cumpre.
3	Implementar a verificação n.º 3 do contrato a sério : comparar a resposta do NEURON com o código base fora da secção atribuída e reprovar se houver alterações.	Fecha a lacuna que a própria especificação diz que os marcadores, sozinhos, só resolvem parcialmente.
4	Unificar a execução no LangGraph : passar o ciclo a correr sobre o grafo compilado, eliminando a reimplementação paralela em <code>MindGraph.run()</code> .	Elimina o risco de as duas descrições do fluxo divergirem.
5	Tornar o parsing de relatórios robusto : pedir saída estruturada aos modelos e tratar o regex apenas como recurso de recurso.	É o primeiro ponto que parte quando entram modelos reais com formato livre.

#	Acção	Porquê agora
6	Piloto com modelos reais via Ollama e preenchimento de docs/model_selection_criteria.md com os resultados dos testes de 5-10 tarefas por componente.	É o passo que valida tudo o resto e desbloqueia o preenchimento dos campos _MODEL e das especialidades dos NEURONS.

Aguardo decisão em dois pontos antes de avançar: se queres que abra um pull request para este ramo, e se concordas com esta ordem de prioridades ou preferes que ataque primeiro outro ponto — por exemplo o piloto com modelos reais, se tiveres endpoints Ollama disponíveis.

Relatório gerado automaticamente no fim da sessão de trabalho. Todas as verificações descritas na secção 3.2 foram executadas nesta sessão; as limitações da secção 4.2 são reportadas tal como existem no código entregue.